Chapter 5: Modelling with Classes

5.0 UML and Diagram Types

The Unified Modelling Language (UML) provides graphical notation for modelling software. The lecture credits Booch, Rumbaugh, and Jacobson and describes OMG standardization beginning in 1997.

- Class diagrams show classes and structural relationships.
- Sequence and communication diagrams show interactions among objects.
- State and activity diagrams describe behaviour and internal flows.
- Component and deployment diagrams show logical organization and physical placement.

5.1 Essentials of Class Diagrams

- A class is a named rectangle, optionally divided into attribute and operation compartments.
- Attributes describe simple data. Associations describe relationships between instances. Generalizations describe inheritance.
- Operation signature: operationName(parameterName: ParameterType): ReturnType.
- Show details appropriate to the model's purpose. Domain models need not contain every implementation method.

5.2 Associations and Multiplicity

An association describes a structural relationship between instances. A link is one instance of that relationship.

5.2.0 Reading multiplicity

Read the multiplicity at one end as the number of objects at that end that may be linked to one object at the opposite end.

- 1 means exactly one; 0..1 means optional; * or 0..* means zero or more; 1..* means one or more. A range gives explicit lower and upper bounds.
- Employee to Company: each employee works for one company in the example's scope; a company can have zero or more employees.
- Company to BoardOfDirectors: each company has exactly one board and each board belongs to exactly one company.

- Passenger to Booking: each booking belongs to exactly one passenger; a passenger can have zero or more bookings.
- Validate both directions, zero cases, and temporary states. Multiplicity expresses domain rules rather than just typical data.

5.2.1 Names, roles, and navigation

- Clear association and role names explain the relationship and distinguish its ends.
- Reflexive associations connect a class to itself, such as Employee supervising Employee. Roles distinguish supervisor and subordinate.
- Reflexive does not imply symmetric: supervising someone does not mean they supervise you.
- Navigation indicates which objects can access related objects. The lecture uses arrows to restrict direction.
- The slides treat undecorated associations as bidirectional; general UML usage can leave navigability unspecified. State the convention used in a model.

5.2.2 Association classes

- Use an association class when information belongs to the relationship rather than either endpoint independently.
- Study example: Enrolment links Student and CourseOffering and holds the grade for that enrolment. The grade is not an intrinsic property of either endpoint alone.
- UML connects an association class to its association using a dashed line. An explicit intermediary class can also represent relationship information.
- Avoid unnecessary one-to-one classes when the information has no distinct identity, lifecycle, or responsibility.

5.3 Generalization

- A hollow triangular arrow points from subclass to superclass. Apply the is-a test and check that inherited behaviour remains meaningful.
- A generalization set groups specializations using a discriminator, the criterion distinguishing subclasses.
- Do not turn individual objects or simple attribute values into subclasses without a structural or behavioural reason.

- Multiple classification criteria may suggest additional generalization levels, multiple inheritance where supported, or separate role objects.
- When a person's role can change, separate the role rather than requiring an existing object to change its runtime class.

5.3.0 Choosing inheritance or a role

A SavingsAccount can specialize Account when every savings account supports the account contract. In contrast, Student and Employee may be changing roles of a Person. A person can hold both roles, so separate role objects can express this without changing the Person object's class.

- A discriminator explains a classification dimension, such as account type. Multiple dimensions need separate consideration rather than a hierarchy containing every possible combination.
- Generalization expresses what an object is. An association expresses what it knows, belongs to, or is linked to.

5.4 Object Diagrams

- Object diagrams show snapshots of particular instances, attribute values, and links.
- An underlined label commonly has the form objectName:ClassName; an unnamed instance can use :ClassName.
- Links instantiate associations. Class-level generalization arrows do not appear between objects.
- A subclass instance is also an instance of its superclasses. It does not require a separate superclass object linked to it.

5.5 Aggregation, Composition, and Interfaces

5.5.0 Whole-part relationships

- Shared aggregation uses a hollow diamond at the whole end. Document the intended ownership meaning.
- Composition uses a filled diamond at the whole end. A part belongs to at most one composite at a time; parts still owned by a destroyed composite are destroyed with it.
- Holding a reference alone does not imply composition. Consider ownership and lifecycle rules.
- Propagation occurs when an aggregate implements an operation by calling corresponding operations on its parts. It requires explicit programming, unlike language-supported inheritance.

5.5.1 Interfaces and explanatory notes

- An interface specifies visible behaviour shared by implementing classes, useful for a common contract among unrelated classes.
- UML realization uses a dashed line with a hollow triangle toward the interface; a small circle can represent a provided interface.
- Diagram notes and surrounding prose explain assumptions, constraints, and rationale that graphics alone cannot express clearly.

5.6 Object Constraint Language (OCL)

OCL expresses precise, side-effect-free properties of a model. Boolean constraints express rules that must hold; other OCL expressions can calculate values or navigate objects without changing state.

5.6.0 Context and conditions

- context identifies the type or operation being constrained; self denotes the contextual instance.
- Invariants hold for instances in valid states described by the model.
- Preconditions are assumed before an operation. Postconditions state what must hold afterward; result denotes the returned value.
- Example: context Person inv: self.age >= 0.
- Slide example: context Person::income(): Integer pre: self.age >= 18 post: result < 5000. These are illustrative domain rules, not universal facts about income.

5.6.1 Navigation and types

- Dot navigation accesses properties and query operations.
  Examples: `self.age`, `self.manager`, and `self.stockPrice()`.
- A many-valued role evaluates to a collection. Use `->` for its operations.
  Example: `self.employee->size()`.
- Set is unordered with unique elements; OrderedSet is ordered with unique elements. Bag permits duplicates; Sequence is ordered and permits duplicates.
- The lecture gives both Sequence and OrderedSet descriptions for ordered roles. The appropriate collection type also depends on whether duplicates are permitted.
- Operators include and, or, not, xor, implies, =, and <>. Arithmetic and string expressions can appear within constraints.

5.6.2 Collection operations

- isEmpty() and notEmpty() test for elements; size() counts them; includes(x) and excludes(x) test membership; sum() totals numeric elements.
- exists(x — condition) requires at least one element satisfying the condition.
- forAll(x — condition) requires every element to satisfy it. It is true for an empty collection, so also require notEmpty() when at least one element is necessary.
- select(x — condition) retains matching elements; reject(x — condition) removes them.
- includesAll(c) and excludesAll(c) compare membership against another collection.
- Set intersection keeps common elements, union combines elements, and excluding(x) removes an element. first() accesses the first element of a nonempty ordered sequence.

5.6.3 Example constraints

- context Company inv: self.employee->notEmpty() requires at least one employee.
- context Company inv: self.employee->select(p — p.age > 50)->notEmpty() requires at least one employee older than 50.
- context Company inv: self.employee->forAll(p — p.forename = 'Jack') requires every employee's forename to be Jack.
- context Company inv: self.employee->forAll(e1, e2 — e1 <> e2 implies e1.forename <> e2.forename) requires distinct employees to have distinct forenames. The guard excludes self-comparison.

5.6.4 Practice rules from the lecture

- Supervisor age and salary comparisons must account for an absent supervisor when the relationship is optional.
- A department manager must belong to that department's employees; employee identifiers must be unique.
- A project's location must be among its department's locations, and an employee cannot supervise itself.
- To limit workers below an hours threshold to two, select those work assignments and require the selected collection's size to be at most two. Hours belong to the work relationship when employees work on multiple projects.

5.6.5 Worked solutions to the practice rules

The following solutions use explicit example role names: Employee has an optional supervisor, age, salary, and ssn; Department has employees, manager, and locations; Project has department, location, and assignments; each WorkAssignment has employee and hours. Match these names to the roles in the class diagram before applying the constraints.

When a supervisor exists, the supervisor must be older and earn at least as much. The conditional avoids navigating an absent supervisor.

```
context Employee inv SupervisorAge:
  if self.supervisor.oclIsUndefined() then true
  else self.supervisor.age > self.age endif

context Employee inv SupervisorSalary:
  if self.supervisor.oclIsUndefined() then true
  else self.salary <= self.supervisor.salary endif

context Employee inv NoSelfSupervision:
  if self.supervisor.oclIsUndefined() then true
  else self.supervisor <> self endif
```

Assuming every department has exactly one manager, the manager must be included in its employee collection. Employee identifiers must be unique over the model's employee population.

```
context Department inv ManagerIsEmployee:
  self.employees->includes(self.manager)

context Employee inv UniqueSSN:
  Employee.allInstances()->isUnique(e | e.ssn)
```

The identifier rule assumes ssn is required. Uniqueness and presence are different requirements: uniqueness alone does not establish that each employee has a valid identifier.

A project's location must belong to its department's locations. At most two employees may work fewer than ten hours on that project. Here each employee has at most one assignment per project, so counting assignments also counts employees.

```
context Project inv AllowedLocation:
  self.department.locations->includes(self.location)

context Project inv OneAssignmentPerEmployee:
  self.assignments->isUnique(a | a.employee)

context Project inv AtMostTwoShortAssignments:
  self.assignments->select(a | a.hours < 10)->size() <= 2
```

If the model permits several assignments per employee on one project, first define whether the ten-hour limit applies to an individual assignment or the employee's total project hours. The constraint must follow that domain rule.

5.7 Developing Class Diagrams

5.7.0 Model scope

- An exploratory domain model helps understand the field.
- A system domain model captures domain information represented by the application.
- A complete system model adds UI, architecture, and utility classes. Keep domain logic independent of a particular UI.

5.7.1 Iterative process

- Identify candidate classes, starting with one or two central ones.
- Add clear attributes and associations, specify multiplicities, and identify useful generalizations.
- Expand to other classes, allocate responsibilities, and identify operations and interfaces.
- Review and iterate, adding or removing elements as understanding improves. Consider reuse and relevant design patterns.

5.7.2 Finding classes and relationships

- Extract nouns and noun phrases from requirements; reject duplicates, instances, vague concepts, and information outside scope.
- Some rejected nouns become attributes. Simple values usually belong in attributes; entities with identity and relationships may require classes.
- Associations describe structural relationships such as membership, ownership, or connection.
- Actions generally become operations. In a library, borrow creates a Loan, while return changes its returned date; these actions should not simply become permanent associations.
- Avoid duplicate attributes. A coherent group of information may justify a separate class.
- Find generalizations upward from similar classes or downward from general concepts. Prefer an interface for a small shared contract among otherwise different classes.

5.7.3 Responsibilities and operations

- A responsibility is work the system must perform. Assign each functional requirement to appropriate collaborating classes.
- Keep responsibilities related. Split overloaded classes, reconsider classes with no responsibilities, and add a class when no existing one fits a required responsibility.

- Responsibilities include creation, attribute access, persistence, linking, searching, computation, transformation, and output.
- Use cases and action verbs help identify responsibilities. Paper cards can record names, attributes, and responsibilities while relationships are rearranged.
- Public operations provide needed services; supporting methods use the narrowest practical visibility.

5.7.4 Collaboration patterns

- Adding a bidirectional link updates both objects consistently.
- Creating a linked object initializes its link and updates the existing object's reference or collection.
- Creating an association object connects it to both endpoints and updates their inverse links.
- Reassigning a link removes the old inverse link and establishes the new one.
- Searching an association traverses related objects and compares the relevant property.

5.8 Implementing Class Diagrams in Java

- Attributes become fields; generalization becomes extends; realization becomes implements.
- A 1 or 0..1 association end commonly becomes an object reference; a many-valued end commonly becomes a collection.
- References do not automatically enforce required multiplicity; collections do not automatically enforce minimum size or uniqueness. Constructors and update methods must preserve constraints.
- Bidirectional relationships usually need fields at both ends and controlled updates to keep them consistent.
- In the lecture example, RegularFlight.addSpecificFlight creates a SpecificFlight referring back to that RegularFlight, then adds it to the collection.
- Restrict low-level constructors and helper operations so callers cannot create invalid partial relationships.

5.8.0 Worked Java example: a one-to-many association

This study example implements the lecture's RegularFlight–SpecificFlight relationship. A regular flight may have zero or more specific flights, and each specific flight belongs to exactly one regular

flight. The factory method establishes both directions. The private constructor prevents clients from bypassing that method.

```
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public final class RegularFlight {
    private final List<SpecificFlight> flights = new ArrayList<>();

    public SpecificFlight addSpecificFlight(LocalDate date) {
        SpecificFlight flight = new SpecificFlight(date, this);
        flights.add(flight);
        return flight;
    }

    public List<SpecificFlight> getSpecificFlights() {
        return List.copyOf(flights);
    }

    public static final class SpecificFlight {
        private final LocalDate date;
        private final RegularFlight regularFlight;

        private SpecificFlight(LocalDate date,
                               RegularFlight regularFlight) {
            this.date = Objects.requireNonNull(date);
            this.regularFlight =
                Objects.requireNonNull(regularFlight);
        }

        public LocalDate getDate() { return date; }

        public RegularFlight getRegularFlight() {
            return regularFlight;
        }
    }
}
```

- The single reference enforces one owner after construction; the collection supports many specific flights. An empty collection satisfies the zero lower bound.
- The returned list is an unmodifiable snapshot, so callers cannot remove links through the getter.
- This example keeps the owner fixed. If reassignment is required, an operation must update the old owner's collection, the new owner's collection, and the reference together.
- Required references do not by themselves imply UML composition. Composition additionally requires the model's ownership and destruction rules.

5.9 Difficulties and Risks

- Strong programming skills do not automatically produce good domain models; choosing abstractions and responsibilities is a separate skill.
- Train the team, involve an experienced modeller, and review diagrams against requirements and concrete scenarios.
- Check multiplicities, role names, inheritance, lifecycle assumptions, and whether implementation operations preserve constraints.